Smart-Attend Mobile Attendance System

Course: SDLC & DevOps Fundamentals
Role: Junior Project Manager

---

1. Project Background

Smart-Attend is a mobile-based attendance management system designed to simplify the process of tracking student presence in classrooms. The application uses **Geo-fencing technology**, which allows the system to automatically detect when a student enters a predefined classroom area. Once the student is detected within the location boundary, the system marks their attendance automatically.

During the initial design stage, the application relied only on location verification. However, during the development phase, the university administration introduced an additional requirement: **Biometric Face ID authentication must be integrated to prevent proxy attendance**.

Proxy attendance occurs when one student attempts to mark attendance on behalf of another. To prevent this issue, the system now needs to verify two factors:

- The **student's physical location** using Geo-fencing
- The **student's identity** using Face ID verification

This change significantly impacts the development process, especially if the project follows the traditional **Waterfall development model**.

---

2. Understanding the Waterfall Development Model

The Waterfall model is a traditional **Software Development Life Cycle (SDLC)** approach where development progresses through a sequence of well-defined phases.

Typical phases include:

1. Requirements Analysis
2. System Design
3. Implementation (Coding)
4. Testing
5. Deployment
6. Maintenance

In this model, each stage must be completed before the next phase begins. Returning to earlier stages is difficult and usually requires major effort.

The Waterfall approach works best when project requirements are **clearly defined and unlikely to change**. However, the introduction of a new biometric feature in the Smart-Attend system creates serious challenges.

---

3. Major Issues with the Waterfall Model

3.1 Limited Flexibility in Handling Changes

One of the most significant weaknesses of the Waterfall model is its limited flexibility. Once the development team completes the requirements and design phases, modifying those decisions becomes difficult.

In the Smart-Attend project, the original system was designed only for **location-based attendance detection**. The system architecture, database structure, and user interface were all planned around that single functionality.

When the requirement for **Face ID authentication** was introduced, several aspects of the system needed revision:

- Updating the requirements documentation
- Redesigning authentication mechanisms
- Adding camera access and biometric processing features
- Updating the application interface for face verification

Because the Waterfall model assumes stable requirements, these modifications disrupt the development process and slow progress significantly.

3.2 Increased Cost and Development Effort

Another critical issue with the Waterfall model is the high cost of implementing changes during later stages of development.

Adding Face ID authentication affects multiple components of the Smart-Attend system.

Mobile Application Changes

The mobile application must interact with the device camera to capture facial images. This requires new interface screens, camera permissions, and proper error-handling mechanisms.

Backend System Changes

The server must process biometric verification requests. This may require integrating a facial recognition engine or using third-party biometric APIs.

Database Adjustments

The database structure must support authentication records, verification logs, and secure storage of identity information.

Security Requirements

Since biometric data is highly sensitive, additional security mechanisms such as encryption, secure communication channels, and strict access control policies must be implemented.

Implementing these changes during the development phase significantly increases both **development time and project cost**.

3.3 Late Discovery of Technical Problems

In the Waterfall model, system testing usually occurs after most of the implementation work has already been completed.

For a feature such as biometric authentication, this approach is risky because several technical challenges may arise:

- Variations in camera performance across different devices
- Face recognition accuracy under different lighting conditions
- Slow authentication due to network delays
- Device compatibility issues

If such problems are discovered only during the final testing stage, developers may need to modify previously completed modules. This leads to additional rework and delays the final product release.

---

4. Overall Impact on the Smart-Attend Project

If the development team continues using the Waterfall model after introducing the biometric requirement, the project may face several negative consequences.

These include:

- Delays in the project timeline due to system redesign
- Increased development costs caused by architectural changes
- Greater risk of software defects due to late integration testing
- Reduced ability to adapt to future changes in requirements

Such limitations can significantly reduce the efficiency of the development process and affect the overall quality of the system.

---

5. Transition to Agile Development (Scrum Framework)

To effectively handle changing requirements, the project team can adopt an **Agile development approach**, particularly the **Scrum framework**.

Agile development focuses on iterative progress, where the project is divided into short development cycles called **Sprints**. Each sprint delivers a small but functional part of the system.

This approach allows the team to adapt quickly to new requirements and continuously improve the application.

---

6. Sprint Planning for the Smart-Attend System

The project can be divided into two **two-week sprints**.

Sprint 1 – Minimum Viable Product (MVP)

Objective: Develop the core functionality of the application.

Key tasks include:

- Designing the basic mobile user interface
- Implementing student login using university credentials
- Developing the Geo-fencing detection mechanism
- Creating the automatic attendance marking system
- Implementing basic notifications for attendance confirmation

At the end of Sprint 1, the team delivers a **working prototype** that demonstrates the core attendance tracking functionality.

Sprint 2 – Feature Enhancement and Security

Objective: Implement biometric verification and improve system reliability.

Key tasks include:

- Integrating Face ID authentication
- Enhancing security mechanisms for identity verification
- Improving the teacher dashboard and reporting features
- Refining the system based on feedback from the first sprint

By the end of Sprint 2, the application becomes a **secure and reliable attendance system** capable of preventing proxy attendance.

---

7. Sprint 1 Backlog Example

Task ID	User Story	Priority	Estimated Hours
US01	As a student, I want to log in using my university ID so that I can access my attendance information.	High	4
US02	As a teacher, I want to see a real-time list of students present in the classroom.	High	6
US03	As a system, I want to detect when a student is within the classroom boundary using Geo-fencing.	High	8

Task ID	User Story	Priority	Estimated Hours
US04	As a student, I want to receive a notification after my attendance is recorded.	Medium	3
US05	As an administrator, I want the ability to manually adjust attendance records if technical issues occur.	Low	4

---

8. Conclusion

The Smart-Attend project highlights the challenges of using a rigid development model in an environment where requirements can evolve.

While the Waterfall model works effectively when project requirements remain stable, it struggles to accommodate major changes during development. The introduction of biometric authentication in the Smart-Attend system exposes the limitations of this approach.

By adopting the Agile methodology and the Scrum framework, the development team can respond to changing requirements more efficiently. Iterative development cycles allow new features such as Face ID verification to be integrated gradually while maintaining system stability and continuous improvement.

Ultimately, this approach results in a **more flexible, secure, and user-friendly attendance management system**.